

中山大学计算机学院

人工智能

本科生实验报告

(2022 学年春季学期)

课程名称: Artificial Intelligence

教学班级	20240574	专业 (方向)	计算机科学与技术
学号	23320093	姓名	林宏宇

一、实验题目

尝试使用 A* 与 IDA* 算法解决 15-Puzzle 问题, 启发式函数可以自己选取, 最好多尝试几种不同的启发式函数

二、实验内容

1. 算法原理

1. A* 算法原理

A* 算法是一种启发式搜索算法, 结合了广度优先搜索和贪心搜索的优点。它通过以下公式计算每个状态的优先级:

$$f(n) = g(n) + h(n)$$

$g(n)$: 从初始状态到当前状态的实际代价。

$h(n)$: 从当前状态到目标状态的启发式估计代价。

$f(n)$: 总代价, 优先队列会优先处理 $f(n)$ 最小的状态。

A* 算法的核心思想是:

使用优先队列存储待扩展的状态, 按照 $f(n)$ 的值从小到大排序。

每次从队列中取出优先级最高, 即启发值 $f(n)$ 最小的状态进行扩展。

如果当前状态是目标状态, 则算法结束, 返回路径。

如果当前状态不是目标状态, 则将其所有可能的后继状态加入队列。

重复上述过程, 直到找到目标状态或队列为空 (无解)。

2. A* 算法在 15-Puzzle 问题中的应用

在 15-Puzzle 问题中:

状态: 拼图的当前排列 (用一维数组表示状态图, 如 (1, 2, 3, ..., 15, 0), 其中 0 表示空格)。

当前状态: 当前的拼图排列为状态序列 (1, 2, 3, ..., 0, 15), 表示图一。

目标状态: 目标的拼图排列为状态序列 (1, 2, 3, ..., 15, 0), 表示图二。



1	2	3	4
5	6	7	8
9	10	11	12
13	14	15	

图一 目标状态

1	2	3	4
5	6	7	8
9	10	11	12
13	14		15

图二 当前状态

启发式函数 $h(n)$: 曼哈顿距离 (每个数字到目标位置的距离之和)。

例子: 当前 1-14 曼哈顿距离都为 0, 15 的曼哈顿距离为 1

实际代价 $g(n)$: 从初始状态到当前状态的移动步数。

例子: 初始时还没移动 $h=0$

A* 算法通过优先队列管理状态, 优先扩展 $f(n) = g(n) + h(n)$ 最小的状态, 从而高效地找到最优解

不断重复拓展的步骤, 直至当前状态和目标状态相同或队列空无解, 则结束。算法输出移动的步数、移动的过程 (上下左右)、算法耗时。

2. 伪代码

function AStar(start_state):

 初始化优先队列 priority_queue

 计算初始状态的曼哈顿距离 g

 初始化 $f = g + h$ (h 为已走步数, 初始为 0)

 将 (f, h, start_state, path) 加入优先队列

 初始化 visited 集合

while priority_queue 不为空:

 从优先队列中取出 f 最小的状态 (f, h, state, path)

 如果 state 已访问过, 跳过

 将 state 标记为已访问

 如果 state 是目标状态:

 返回 path (空格的移动路径)

 遍历 state 的所有邻居 new_state:

 如果 new_state 未访问过:

 计算 $new_h = h + 1$ (已走步数)

 计算 $new_g =$ 曼哈顿距离(new_state)

 计算 $new_f = new_h + new_g$

 将 (new_f, new_h, new_state, path + [move]) 加入优先队列

返回 None (无解)



function Manhattan(state):

 初始化 distance = 0

 遍历 state 中的每个元素 val 和索引 i:

 如果 val == 0, 跳过

 计算目标位置的坐标 (target_x, target_y)

 计算当前坐标 (current_x, current_y)

 累加曼哈顿距离: distance += abs(target_x - current_x) + abs(target_y - current_y)

 返回 distance

function GetNeighbors(state):

 找到空格 (0) 的索引 zero_index

 初始化 neighbors 列表

 遍历每个可能的移动方向 (move, delta):

 计算新的索引 new_index = zero_index + delta

 如果 new_index 在边界内:

 交换 zero_index 和 new_index 的值, 生成 new_state

 将 (move, new_state) 加入 neighbors

 返回 neighbors

主程序:

 初始化初始状态 initial_state

 调用 AStar(initial_state)

 如果有解:

 输出路径、步数和运行时间

 否则:

 输出 "无解"

3. 关键代码展示（带注释）

1. 移动操作的表示

#允许的移动方向（左、右、上、下）

#比如 "L": -1 表示向左移动，在一维数组中索引-1，在二维数组中表示向左移动一格

```
MOVES = {
```

```
    "L": -1, "R": 1,
```

```
    "U": -4, "D": 4
```

```
}
```

2. 曼哈顿距离的计算公式

#计算曼哈顿距离

```
def manhattan(state):
```

```
    distance = 0
```

```
    for i, val in enumerate(state):
```



```
if val == 0: #空格不计算曼哈顿距离
    continue

target_x, target_y = (val - 1) % 4, (val - 1) // 4

#对一维数组的索引: %4 取模得到横坐标, //4 整除得到纵坐标

current_x, current_y = i % 4, i // 4

distance += abs(target_x - current_x) + abs(target_y - current_y)

#曼哈顿距离公式 = abs(目标 x - 当前 x) + abs(目标 y - 当前 y)
```

```
return distance
```

3. 拓展下一个可能状态的函数

#拓展当前状态发生移动后的所有可能的状态

```
def get_neighbors(state):

    zero_index = state.index(0) #找到空格的位置, 空格 0 可以进行上下左右移动

    neighbors = []

    for move, delta in MOVES.items():

        new_index = zero_index + delta

        new_x, new_y = new_index % 4, new_index // 4 #得到新的空格坐标

        if 0 <= new_x < 4 and 0 <= new_y < 4: #判断是否在边界内, 防止 0 移动到边界外

            new_state = list(state)

            new_state[zero_index], new_state[new_index] = new_state[new_index], new_state[zero_index]

            neighbors.append((move, tuple(new_state)))

    return neighbors
```

4. A*算法

#A* 算法求解

```
def solve_puzzle(start_state):

    priority_queue = []

    h=0 #已走步数, 表示初始状态到当前状态代价

    g=manhattan(start_state) #计算曼哈顿距离, 表示当前状态到目标状态的代价

    f=g+h #启发函数 f=曼哈顿距离 g+已走步数 h

    heapq.heappush(priority_queue, (f, h, start_state, []))

    #将元素加入优先队列 (最小堆)

    #最小堆函数 输入参数为: 曼哈顿距离, 已走步数, 当前状态, 路径

    #优先比较输入的第二个参数, 最小的在前面

    visited = set()

    while priority_queue:

        f, h, state, path = heapq.heappop(priority_queue) #取出启发值 f 最小的情况

        if state in visited: #如果当前状态已经访问过, 则跳过

            continue
```

```
visited.add(state)

if state == GOAL_STATE: #如果当前状态等于目标状态，则返回路径
    return path

for move, new_state in get_neighbors(state): #拓展当前状态的所有可能的状态
    if new_state not in visited:
        new_h = h + 1 #记录已走步数
        new_g = manhattan(new_state) #计算新的曼哈顿距离
        new_f = new_h + new_g # 计算 f=h+g
        heapq.heappush(priority_queue, (new_f, new_h, new_state, path + [move]))

return None #队列为空 无解
```

三、 实验结果及分析

1. 实验结果展示示例（可图可表可文字，尽量可视化）

样例一：

1	2	4	8
5	7	11	10
13	15		3
14	6	9	12

最优路径下空格的移动方向: L->D->R->U->R->U->L->D->R->U->U->L->D->L->D->
D->L->U->R->R->R->D

移动步数: 22

移动时间: 0.0070023537 秒

样例二：

14	10	6	0
4	9	1	8
2	3	5	11
12	13	7	15

最优路径下空格的移动方向: R->R->R->U->L->D->D->L->U->U->R->D->D->R->R->
->R->U->R->R->U->U->L->D->L->D->D->R->U->U->R->D->D->L->U->U->R->
D->D->R

移动步数: 39

移动时间: 6.2397618294 秒



样例三：

5	1	3	4
2	7	8	12
9	6	11	15
	13	10	14

最优路径下空格的移动方向: R->R->R->U->U->L->L->L->U->R->D->D->D->R->R

移动步数: 15

移动时间: 0.0010001659 秒

样例四：

6	10	3	15
14	8	7	11
5	1		2
13	12	9	4

最优路径下空格的移动方向: R->U->R->R->U->L->D->L->L->D->L->U->U->L->D->D->L->U->L->D->D->D->L->U->R->R->U->R->R->R->R->D->L->U->U->R->D->L->L->U->U->R->R->D->D->D

移动步数: 46

移动时间: 172.8523983955 秒

样例五：

11	3	1	7
4	6	8	2
15	9	10	13
14	12	5	

最优路径下空格的移动方向: L->L->L->U->R->D->L->L->D->L->L->U->R->U->R->D->L->L->U->R->U->L->D->R->R->U->R->R->R->U->L->L->D->R->U->R->D->L->D->R->U->R->D->D

移动步数: 44

移动时间: 48.6903033257 秒

样例六：

	5	15	14
7	9	6	13
1	2	12	10

8

11

4

3

最优路径下空格的移动方向: D->D->L->U->L->L->L->D->R->D->D->L->U->L->D->
D->L->L->U->L->L->U->L->L->D->R->R->D->D->L->L->U->L->U->L->D->D->
D->L->U->U->U->R->D->D->D->L->U->R->D

移动步数: 50

移动时间: 149.8993983269 秒

2. 评测指标展示及分析

1. 移动步数分析

步数与初始状态: 不同的初始状态会导致不同的最优路径和步数, 样例中步数的范围是从 15 步到 50 步不等。步数较少的情况 (如样例三, 15 步) 往往代表初始状态较为接近目标状态, 而步数较多的情况 (如样例五, 50 步) 则意味着初始状态与目标状态差异较大, 需要更多的步骤进行移动。

步数与复杂度: 随着步数的增加, 意味着问题的复杂度也随之增加。这通常与初始状态中的数字分布、空格的位置以及目标状态的配置相关。例如, 样例四 (46 步) 和样例五 (50 步) 中的步数较长, 表明这些状态需要更多的时间和步骤来解决。

2. 移动时间分析

时间与步数: 移动时间通常与移动步数成正比, 步数越多, 解决问题的时间通常也越长。比如样例五, 尽管步数和时间都很高 (50 步与 149.9 秒), 这说明计算过程中可能涉及到更多的搜索或更复杂的路径优化。

效率的考量: 如果程序能够在较短时间内完成较多步数的移动 (如样例一中的 22 步, 移动时间仅 0.007 秒), 则表明算法的执行效率较高。相反, 较长的执行时间 (如样例四中的 172.8 秒) 可能表示算法在解空间探索或路径优化方面的瓶颈。

3. 算法性能与优化方向

路径优化: 在一些样例中, 空格的移动方向是较为直观的 (如样例三), 这可能是由于算法在状态空间中能够快速收敛, 找到最优路径。对于更复杂的状态 (如样例四和样例五), 路径的选择变得更加复杂, 可能需要更多的策略来优化。

时间消耗高的原因: 某些复杂的初始状态导致程序需要进行较多的回溯或状态评估, 这可能是算法本身或搜索方法不够高效。可以考虑采用更高效的启发式搜索算法, 如 A* 或 IDA*, 来减少状态空间的探索时间。

4. 实验结论

从结果来看, 实验中的算法在大多数情况下能够求解出最优路径, 并且移动步数与时间表现出一定的规律性。较为简单的初始状态 (如样例三) 能在较短时间内解决, 而复杂的状态 (如样例四和样例五) 则显示出较长的计算时间和更多的步数。

在实际应用中, 可以进一步优化算法的性能, 尤其是在处理复杂状态时, 减少不必要的计算, 提升效率。



3. 优化方案一的实验结果展示示例

优化方案一：权重系数调节 $f=a*g+b*h$

数学原理：

当 $a=1$ 和 $b=1$ ，意味着 f 会同时考虑实际路径代价和启发式估计，在大多数情况下，能提供较好的平衡。

当 a 较大， b 较小时，算法会更加注重已经走过的路径，从而可能导致较长时间的搜索，但能更好地避免过度依赖不准确的启发式函数。

当 a 较小， b 较大时，算法会更依赖启发式估计，从而加快搜索速度，但可能导致不那么精确的路径。

示例：

初始状态：

(14, 10, 6, 0)

(4, 9, 1, 8)

(2, 3, 5, 11)

(12, 13, 7, 15)

权重系数： $a = 1$ $b = 1$

移动步数： 39

移动时间： 6.2095823288 秒

权重系数： $a = 1.1$ $b = 0.9$

移动步数： 39

移动时间： 118.2289612293 秒

权重系数： $a = 0.9$ $b = 1.1$

移动步数： 41

移动时间： 4.4640562534 秒

权重系数： $a = 0.8$ $b = 1.2$

移动步数： 43

移动时间： 0.7346527576 秒

权重系数： $a = 0.7$ $b = 1.3$

移动步数： 43

移动时间： 0.0232322216 秒

4. 优化方案一的评测指标展示及分析

1. 移动步数

在不同的 a 和 b 设置下，移动步数变化并不明显。相对较高的步数往往出现在启发式权重较大的设置下 ($a < 1, b > 1$)，这表明更依赖启发式估计时，算法可能会选择一个较长的路径，导致需要更多的搜索步骤。

2. 移动时间

移动时间的变化反映了启发式评估在搜索中的重要性：

当 $a = 1, b = 1$ 时，搜索时间较短，这表明算法在平衡路径代价和启发式估计代价时能较好地控制搜索的深度，避免过度依赖不精确的启发式信息。

当 $a = 0.7, b = 1.3$ 时，搜索时间大幅减少。这表明更多依赖启发式信息时，算法能够较快地找到一个近似的解，但可能会牺牲路径的精确性。

3. 应用

当 $a = 0.7, b = 1.3$ 时，搜索时间大幅减少，搜索速度极大提高，应用到下面的示例中。

初始状态：

(6, 10, 3, 15)

(14, 8, 7, 11)

(5, 1, 0, 2)

(13, 12, 9, 4)

权重系数： $a = 1, b = 1$

移动步数： 46

移动时间： 172.8523983955 秒

权重系数： $a = 0.7, b = 1.3$

移动步数： 54

移动时间： 2.6918740273 秒

5. 优化方案二的实验结果展示示例

优化方案二：更换曼哈顿函数 h 为不正确块数函数、切尔雪夫距离函数、线性冲突函数

1. 不正确块数函数

```
#计算不正确块数
def incorrect_tiles(state):
    incorrect_count = 0
    for i, val in enumerate(state):
        if val == 0: #空格不计算在内
            continue
        if val != GOAL_STATE[i]: #如果当前值与目标状态不同
            incorrect_count += 1
    return incorrect_count
```

2. 切尔雪夫距离函数

```
#计算切尔雪夫距离
def chebyshev(state):
    distance = 0
    for i, val in enumerate(state):
        if val == 0: # 空格不计算切尔雪夫距离
            continue
```



```
target_x, target_y = (val - 1) % 4, (val - 1) // 4

# 对一维数组的索引：%4 取模得到横坐标，//4 整除得到纵坐标

current_x, current_y = i % 4, i // 4

# 切尔雪夫距离公式 = max(目标 x - 当前 x, 目标 y - 当前 y)

distance += max(abs(target_x - current_x), abs(target_y - current_y))

return distance
```

3.线性冲突函数

#线性冲突启发式函数

```
def calculate_heuristic(state):

    manhattan_distance = 0 # 曼哈顿距离

    linear_conflict = 0 # 线性冲突

    for row in range(4):

        for col in range(4):

            value = state[row * 4 + col]

            if value != 0:

                # 计算目标位置的行和列

                target_row, target_col = (value - 1) // 4, (value - 1) % 4

                manhattan_distance += abs(row - target_row) + abs(col - target_col)

                # 计算行冲突

                if col == target_col:

                    for next_col in range(col + 1, 4):

                        next_value = state[row * 4 + next_col]

                        if next_value != 0 and (next_value - 1) // 4 == row and (next_value - 1) % 4 < col:

                            linear_conflict += 2

                # 计算列冲突

                if row == target_row:

                    for next_row in range(row + 1, 4):

                        next_value = state[next_row * 4 + col]

                        if next_value != 0 and (next_value - 1) % 4 == col and (next_value - 1) // 4 < row:

                            linear_conflict += 2

    return manhattan_distance + linear_conflict
```

示例展示：

示例一：

初始状态：

(1, 2, 4, 8)

(5, 7, 11, 10)

(13, 15, 0, 3)

(14, 6, 9, 12)



h 为曼哈顿距离

移动步数: 22

移动时间: 0.0070023537 秒

h 为不正确块数

移动步数: 22

移动时间: 1.1274347305 秒

h 为切尔雪夫距离

移动步数: 22

移动时间: 0.0880067348 秒

h 为线性冲突

移动步数: 22

移动时间: 0.0146739483 秒

示例二:

初始状态:

(5, 1, 3, 4)

(2, 7, 8, 12)

(9, 6, 11, 15)

(0, 13, 10, 14)

h 为曼哈顿距离

移动步数: 15

移动时间: 0.0010001659 秒

h 为不正确块数

移动步数: 15

移动时间: 0.0019989014 秒

h 为切尔雪夫距离

移动步数: 15

移动时间: 0.0020000935 秒

h 为线性冲突

移动步数: 15

移动时间: 0.0009338856 秒

示例三:

初始状态:

(14, 10, 6, 0)

(4, 9, 1, 8)

(2, 3, 5, 11)

(12, 13, 7, 15)

h 为曼哈顿距离

移动步数: 39

移动时间: 6.2397618294 秒

h 为不正确块数

移动步数: 39

移动时间: 596.5070490837 秒

h 为切尔雪夫距离

移动步数: 39

移动时间: 611.3269503117 秒

h 为线性冲突

移动步数: 39

移动时间: 3.2757456303 秒

6. 优化方案二的评测指标展示及分析

1. 各种启发式函数的搜索时间差异较大，具体表现如下：

曼哈顿距离：通常能够在较短时间内完成搜索，这表明曼哈顿距离是一种较为高效的启发式评估方式，能够较好地估计到目标状态的代价，适合用于类似八数码问题这类局部搜索。

不正确块数：尽管该启发式方法较为简单，但它的效率较低，导致搜索时间显著增加。这表明虽然不正确块数是一个有效的启发式函数，但它可能缺乏对目标状态的精确估计，导致搜索过程更为漫长。

切尔雪夫距离：相比于曼哈顿距离，切尔雪夫距离能够提供更加精确的估计，但由于需要计算水平和垂直方向的最大距离，这导致它的计算时间稍长，搜索时间比曼哈顿距离长，但与不正确块数相较稍好。

线性冲突：线性冲突作为一种更复杂的启发式方法，能够显著提高路径估计的精度，尤其对于相邻块的顺序会产生较大影响。尽管线性冲突的计算较为复杂，但它通常能够较快地搜索到目标状态，搜索时间也较短。

3. 影响因素分析

启发式函数对搜索时间有显著影响，尤其在计算更精确的估计时，尽管可以加速找到目标状态，但同时也会导致计算成本的增加。

对于计算代价较低的启发式函数（如曼哈顿距离），搜索时间较短，但可能缺乏对目标状态的精确估计。

线性冲突和切尔雪夫距离相较于其他启发式方法更为复杂，计算量较大，因此在时间上可能会有所增加，但也能带来更精确的路径估计。

4. 优化效果

简单启发式（如曼哈顿距离）：适用于大多数简单问题，能够在较短时间内找到解。



复杂启发式（如切尔雪夫距离和线性冲突）：适用于更为复杂的状态空间，能够提供更精确的路径估计，但搜索时间较长。

对于不同的启发式函数，选择应根据具体问题的复杂度和计算资源进行平衡。

四、 实验题目

IDA*算法解决 15-Puzzle 问题

五、 实验内容

1. 实验原理

IDA*是一种结合了深度优先搜索和 A*搜索的算法。它通过逐步增加搜索的深度限制来进行状态空间的搜索，同时利用启发式函数来优化每一步的搜索顺序。IDA 的目标是避免像 A* 那样需要大量存储中间状态如维护优先队列，而是通过递归和深度限制的方式有效节省内存。

IDA*的核心思想：

深度优先搜索：IDA*每次都深度优先地扩展状态空间，从根节点起始状态开始向下搜索。

A*启发式函数：每次深度优先搜索时，使用 A 的启发式函数来决定搜索优先级。A 的启发式函数通常定义为： $f(n) = g(n) + h(n)$ ，其中：

$g(n)$ 是从起点到当前节点的实际代价。

$h(n)$ 是当前节点到目标节点的启发式估计代价。

迭代加深：与传统的深度优先搜索不同，IDA*在每一轮搜索中都会设定一个深度限制 bound，并执行一次深度优先搜索。当搜索结束时，如果没有找到目标状态，它会增加搜索的深度限制，重新从根节点开始搜索，直到找到解为止。

2. 伪代码

函数 manhattan(状态):

初始化 distance = 0

对于 每个元素 (索引 i, 值 val):

 如果 val != 0:

 计算目标位置 (target_x, target_y)

 计算当前坐标 (current_x, current_y)

 更新 distance += abs(target_x - current_x) + abs(target_y - current_y)

返回 distance

函数 get_neighbors(状态):

 获取空格位置 zero_index

 对每个可能的移动方向 (move, delta):

 计算新位置 new_index = zero_index + delta

 如果新位置在有效范围内，交换位置并返回新状态

 返回所有邻居状态

函数 ida_star(起始状态 start_state):

 初始化 bound = manhattan(起始状态)

定义递归函数 `search`(当前状态 `state`, 走过的步数 `g`, 当前路径 `path`, 当前深度限制 `bound`):

```
f = g + manhattan(state)
如果 f > bound 返回 f
如果 state == GOAL_STATE 返回路径 path
min_bound = ∞
对每个邻居 (move, new_state):
    如果 new_state 不在 path:
        result = search(new_state, g + 1, path + [move], bound)
        如果 result 是路径 返回 result
        更新 min_bound = min(min_bound, result)
返回 min_bound
```

循环:

```
result = search(start_state, 0, [], bound)
如果 result 是路径 返回 result
如果 result == ∞ 返回 None
更新 bound = result
```

主程序:

```
初始化 initial_state
开始计时
solution = ida_star(initial_state)
结束计时
如果 solution:
    打印路径和步数
否则:
    打印 "该状态无解"
```

3. 核心代码展示

这里仅展示与 A*算法不同的代码

```
#IDA*算法求解
def ida_star(start_state):
    def search(state, g, path, bound, visited):
        f = g + manhattan(state) #启发函数 f
        if f > bound: #若超过当前深度限制, 返回 f 值
            return f
        if state == GOAL_STATE: # 到目标状态
            return path
        min_bound = float('inf')
```



```
visited.add(state) #将当前状态加入访问过的集合

for move, new_state in get_neighbors(state):

    if new_state not in visited: #防止循环，检查新状态是否已访问

        new_path = path + [move]

        result = search(new_state, g + 1, new_path, bound, visited)

        if isinstance(result, list): #如果找到路径 isinstance 函数判断是否为某一类型

            return result

        min_bound = min(min_bound, result)

visited.remove(state) #移除当前状态，回溯时允许访问

return min_bound


bound = manhattan(start_state) #初始深度限制等于起始状态的曼哈顿距离

while True:

    visited = set() #用集合来存储访问过的状态

    result = search(start_state, 0, [], bound, visited) #result 是路径或深度限制

    if isinstance(result, list): #如果找到解，返回的是路径

        return result

    if result == float('inf'): #如果没有解，返回的是无穷大

        return None

    bound = result #更新深度限制
```

六、 实验结果及分析

1. 实验结果展示示例（可图可表可文字，尽量可视化）

样例一：

1	2	4	8
5	7	11	10
13	15		3
14	6	9	12

A*

移动步数： 22

移动时间： 0.0070023537 秒

IDA*

移动步数： 22

移动时间： 0.0070130825 秒



样例二：

14	10	6	0
4	9	1	8
2	3	5	11
12	13	7	15

A*

移动步数： 39

移动时间： 6.2397618294 秒

IDA*

移动步数： 39

移动时间： 5.6807231903 秒

样例三：

5	1	3	4
2	7	8	12
9	6	11	15
	13	10	14

A*

移动步数： 15

移动时间： 0.0010001659 秒

IDA*

移动步数： 15

移动时间： 0.0000000000 秒

样例四：

6	10	3	15
14	8	7	11
5	1		2
13	12	9	4

A*

移动步数： 46

移动时间： 172.8523983955 秒

IDA*

移动步数: 46

移动时间: 268.1297221184 秒

样例五:

11	3	1	7
4	6	8	2
15	9	10	13
14	12	5	

A*

移动步数: 44

移动时间: 48.6903033257 秒

IDA*

移动步数: 44

移动时间: 77.3219685555 秒

样例六:

	5	15	14
7	9	6	13
1	2	12	10
8	11	4	3

A*

移动步数: 50

移动时间: 149.8993983269 秒

IDA*

移动步数: 50

移动时间: 303.0707478523 秒

2. 评测指标展示及分析

1. 移动时间

A*算法通常在搜索过程中效率较高, 因为它的启发式评估(如曼哈顿距离)能够有效引导搜索方向, 从而减少不必要的状态探索。移动时间相对较短, 尤其是在样例三中, 移动时间仅为 0.001 秒。

IDA 算法通常需要反复进行深度限制的调整, 因此在一些复杂的状态空间中, 可能需要

更多的时间来进行深度搜索和回溯，导致其移动时间较 A 略长，尤其在样例四、样例五和样例六中，IDA 的时间明显高于 A。这种现象尤其在状态空间较大时更为明显。

2. 移动时间的对比

样例三：A 和 IDA 的移动步数都相同，但 IDA 的移动时间几乎为 0，这可能表明 IDA 通过深度优先的策略快速找到了解，减少了状态扩展的时间开销，尽管没有明显的时间消耗。

样例四和样例五：IDA 的移动时间显著长于 A，特别是在样例四，IDA 的移动时间几乎是 A 的两倍。这是因为 IDA 的深度限制不断更新，导致多次递归调用，在较复杂的状态下，IDA 的计算量大，性能受限。

3. 总结与分析

A* 优点是搜索较快，适合解决小规模或较简单的搜索问题，尤其在状态空间较小的情况下能够高效运行。通过启发式函数引导搜索，减少了无效扩展的状态，通常表现出较低的计算开销。

IDA*：通过深度优先的方式，在内存开销上更具优势（避免存储整个搜索树），但在一些复杂状态下，可能会导致较长的计算时间，尤其在深度限制频繁更新时，性能可能受到影响。因此，IDA 适合内存受限的场景，但对于时间要求较高的应用，A 可能更具优势。

3. 优化部分更新启发式函数，同 A* 算法，不再重复

七、 思考题

1. 启发函数的选择

启发式函数的作用是指导搜索引擎沿着最有可能找到解的路径前进。它是 A 和 IDA 算法中最核心的组成部分之一。启发式函数不能超过实际代价，同时启发式函数不应低估代价。不同的启发式函数对搜索性能和结果的准确性有着重要影响。

2. A* 与 IDA* 算法的对比

A* 优点是搜索较快，适合解决小规模或较简单的搜索问题，尤其在状态空间较小的情况下能够高效运行。通过启发式函数引导搜索，减少了无效扩展的状态，通常表现出较低的计算开销。

IDA*：通过深度优先的方式，在内存开销上更具优势（避免存储整个搜索树），但在一些复杂状态下，可能会导致较长的计算时间，尤其在深度限制频繁更新时，性能可能受到影响。因此，IDA 适合内存受限的场景，但对于时间要求较高的应用，A 可能更具优势